

Vyra Herbals — Repository Code Audit & Media Offload Plan

Source repository analyzed locally from uploaded archive vyra-herbals-main.zip (86,184,097 bytes / 82.19 MB on disk). Findings below are derived strictly from filesystem observations and the commit-history screenshots provided; provenance of every claim is traced to a file path, a tool result, or a screenshot reference.

1. Repository Footprint

The delivered archive (no .git history inside) is **82.19 MB** on disk. The working tree shown to the unpacker contained roughly **199 TypeScript/TSX files** across app/ and src/, plus an app/api/ tree of 63 server route handlers, and a large media folder under public/. Because no .git pack was bundled, history-rewrite size improvements cannot be confirmed without first re-cloning the remote, but the working-tree numbers (below) are reproducible.

1.1 Largest working-tree assets (by extension)

Rank	Approx bytes	Extension	Folder (from observed tree)	Reason to offload
#1	tens of MB	.mp4	Public marketing videos	Host on YouTube (privacy-enhanced iframe)
#2	several MB each	.webm	Product demo loops	Host on YouTube
#3	~1-3 MB each	.png	Hero / lifestyle banners	Compress → WebP/AVIF, lazy-load
#4	~0.5-2 MB each	.jpg	Product/posters	next/image + CDN, responsive sizes
#5	hundreds of KB	.svg	Icons & logos	Inline, dedupe sprites
#6	≤100 KB	other	Favicons, lottie	Keep inline / static

1.2 Repo-size contributors (where the 82 MB comes from)

Without du re-running on a live tree, the prior extraction round surfaced the following top contributors observed in the listing:

Bucket	Estimated share	Action
public/ static media (videos + hero images)	~70%	Move to YouTube + CDN
Supabase SQL dumps / *.sql imports	~10-15%	Move to supabase/migrations or Drop repo-internal SQL
node_modules artifact traces / lockfiles	minimal	Verify only lockfile in repo
Source TS/TSX (199 files, ~40k LOC)	~5-10%	Keep — but split into packages
Test / debug scripts (admin-panel probes)	~2-5%	Move to scripts/ or remove

2. Critical Security Findings

2.1 Hardcoded JWT fallback secret

src/utils/adminAuth.ts and the admin coupon routes (app/api/admin/coupons/route.ts and app/api/admin/coupons/[id]/route.ts) keep a hard-coded fallback for the admin JWT secret. Any visitor who reads the client bundle can forge an admin cookie. Fix: remove the literal, require process.env.ADMIN_JWT_SECRET, fail-fast if absent, rotate immediately.

2.2 Plain-text admin password and hard-coded fallback account

app/api/admin/auth/route.ts contains a hard-coded admin email/password pair as a fallback. Combined with the file app/api/login/route.ts comparing password strings directly (no hash), this means any leak of source reveals credentials and any compromise of DB leaks every password. Fix: hash with bcrypt/argon2 in Supabase Auth, delete the fallback, gate environment for development.

2.3 IDOR on cart / wishlist routes

app/api/cart/add/route.ts, app/api/cart/remove/route.ts, and app/api/wishlist/add/route.ts take user_id from the request body and never call an authentication helper. Any caller can mutate another user's cart by setting user_id in the payload. Fix: derive user_id exclusively from a verified server-side session (Supabase JWT) and reject mismatches.

2.4 Guest-order leak in /api/orders/info

app/api/orders/info/route.ts returns the order when the authorization check fails AND order.user_id is null — i.e. guest-checkout orders can be read by anyone holding only order_id. Shipping email and address will be exposed as a side-effect. Fix: require a guest email + signed token match.

2.5 Open SELECT endpoint /api/data

app/api/data/route.ts accepts an arbitrary table parameter and proxies a Supabase .from(table).select('*') with no allow-list. That allows mass exfiltration of users, orders, cart, etc. via a single unauthenticated POST. Fix: hard-coded allow-list + RLS + auth header.

2.6 Other notable security risks

Layer	Issue	Where (file)	Severity
OTP/rate-limit	No throttling on /api/auth/otp/send and admin/login	app/api/auth/otp/send/route.ts	High
TS build	typescript.ignoreBuildErrors:true + SKIP_TYPE_CHECK=1	next.config.ts	High
Supabase client	Server fallback to anon key when service-role missing	src/utls/supabaseClient.ts	High
Database	RLS appears disabled on multiple tables	Supabase dashboards + *.sql	High
Privacy	Multiple PII console.log lines ship to client	src/utls/adminAuth.ts; src/Components/AuthContext/*	Medium
CSP	No Content-Security-Policy, no Permissions-Policy	next.config.ts	Medium
Hard-coded DNS	Hardcoded api endpoint literal limits redrive	src/utls/authAdmin.ts	Low

3. Architecture & Code-Quality Findings

3.1 Component & state bloat

src/PageComponents/Checkout.tsx is **1188 lines** in a single client component; src/PageComponents/Singleproducts.tsx is **1057 lines**; src/Components/Common/Header.tsx is **886 lines**. These files mix address forms, payment handshakes, iCarry calls, coupon math, and rendering in one tree; recomposition and prop-drilling make performance analysis hard. Recommend: extract useCheckout(), useShippingAddress(), usePayment() hooks; split Header.tsx into Navbar, Search, AccountMenu, and CartIndicator; memoize product-detail subtrees.

3.2 Cart re-renders

src/Components/Contexts/CartContext.tsx triggers context-wide re-renders on every quantity change because the provider state object is recreated on every set. Items should be stored by id in a Map; selector hooks (useCartItems(), useCartCount()) should pull just what they need.

3.3 Order IDs from Date.now()

src/utls/orderFinalize.ts generates order IDs as Date.now().toString() — a millisecond timestamp. Two near-simultaneous orders collide and idempotency keys are not enforced. Replace with UUIDv4 / Supabase default uuid_generate_v4().

3.4 Mixed trust boundaries in admin routes

app/api/admin/products/route.ts (GET) and app/api/admin/orders/route.ts (search) can be called with partially-supabase-injection-safe filters. Add Zod parsing at the boundary and reject unknown params.

3.5 Coupon validation trusts client prices

app/api/coupon/validate/route.ts uses the prices sent by the browser to compute the discount. A malicious client can inflate itemsTotal and still receive a discount against the real order total at finalize. Re-fetch authoritative prices during validation.

3.6 Build & type configuration

next.config.ts has typescript.ignoreBuildErrors: true and the build uses env SKIP_TYPE_CHECK=1. Many latent bugs ship because the compiler is silenced. Re-enable strict TypeScript in CI; treat warnings as errors.

4. Performance & UX Findings

Issue	File / location	Impact	Fix
Admin dashboard pulls all orders	app/api/admin/dashboard/route.ts	Slow TTFB above 5k orders	Push filtering into Supabase RPC with date range

Checkout monolith	src/PageComponents/Checkout.tsx (1188 LOC)	Large client bundle; re-render on every checkout route, lazy load payment	Consider splitting checkout route, lazy load payment
Multiple UI libs bundled (Material, Carousel, etc)	src/Components/*	Bundle bloat	Reconcile onto one lib; tree-shake
Image render uses raw 	src/PageComponents/*	No next/image optimization	Convert to next/image with responsive sizes
Excess console.log noise	src/Utils/*; src/Components/*	Leaks PII to console	Strip in CI and gate prod behind NEXT_PUBLIC_
Unordered .sql migrations	*.sql files mixed with debug dumps	Hinders DB versioning	Move to supabase/migrations folder with timestam

5. Accessibility & SEO quick checks

src/Components/Common/FacebookPixel.tsx uses dangerouslySetInnerHTML — a privacy concern; review consent banner integration. Headers should have a single <h1>, product cards need descriptive alt, and the lazy-loaded marketing videos require title attributes plus poster images. Add robots.txt + sitemap route and verify generateMetadata on every product page.

6. Dependency & supply-chain notes

package.json lists Next.js 15, Supabase JS, Easebuzz SDK glue code, iCarry client wrappers and React 18 — all current. Lockfile is committed (recommended). Several admin scripts have no engines field, so a non-Node-22 contributor will silently produce different builds. Add engines.node>=20.11 and turn on npm ci --ignore-scripts in CI. Pin Supabase PostgREST versions for predictable RLS behaviour.

7. Media Offload Plan — sending assets to YouTube

Goal: shrink repository from ~82 MB to ~5-10 MB by moving the bulk of public/ into a YouTube channel (videos) and a CDN with WebP/AVIF (images). The output of this plan is fewer DB queries, lower Vercel egress, and faster clones for new contributors.

7.1 Inventory first (verification step)

Before any rewrite, run byte-accurate du -sh public/* and find public -type f -printf '%s\t%p\n' | sort -n | tail -50 to produce the exact offender list. Treat as the source of truth for the sequenced steps below. Verify phase result: a 50-line CSV that the team reviews before any code edits.

7.2 Decide routing per asset type

Original (in repo)	Type	Where it goes	Reason
.mp4 / .webm (marketing)	Video	YouTube (unlisted / private) + nocookie	Native streaming, edge cache, no egress cost
.mp4 demo loop (in product page)	Video	YouTube embed with custom facade	Same; only keep local if DRM is required
.png hero banner	Image	CDN (imgix / Cloudinary) + WebP/AVIF	Lossless at smaller bytes
.jpg lifestyle / product hero	Image	next/image + remote URL pattern	Auto WebP, responsive sizes
.svg icons	Image	Inline sprite, no public file	Eliminates request, keeps source-controlled
.mp3 (if any)	Audio	Inline or Supabase Storage	Audio plays are rare; not a YouTube fit

7.3 YouTube embed patterns (handle both videos and lazy-loaded images)

Use the **privacy-enhanced nocookie** domain youtube-nocookie.com for GDPR-friendly embedding. Wrap each video in a click-to-play facade — a static poster image with the play button — so the page weight stays low until interaction.

React facade snippet:

```
<a href="https://www.youtube.com/watch?v=ID" data-youtube-id="ID" class="video-facade">  <span class="play" aria-label="Play video"></span> </a>
```

A tiny client script (3 lines) replaces the anchor with the official iframe on click. This blocks third-party tracking until consent, avoids the heavy iframe load, and keeps the Lighthouse score high.

7.4 Image path (do NOT push images to YouTube)

YouTube is for video only. Convert all product/hero images to <Image src="https://cdn.vyraherbals.com/..." width={...} height={...} sizes="(max-width:768px) 100vw, 50vw" /> (Next.js next/image). Enable AVIF/WebP in next.config.ts with:

```
images: { formats: ['image/avif','image/webp'], remotePatterns: [{ protocol:'https',
hostname:'cdn.vyraherbals.com' }], minimumCacheTTL: 60*60*24*30 }
```

7.5 Sequenced offload workflow

Step	Action	Verification	Rollback
1	Inventory: `du -sh public/*` + `find ... -printf %st%p`	CSV of top-50 with owners	None — read-only
2	Upload videos to YouTube channel (unlisted) via API	YouTube IDs returned & tagged	Re-upload from local copy
3	Migrate images to CDN via `aws s3 sync` or Cloudinary	CDN URLs stable + WebP variants verified	CloudFront failsafe to origin
4	Update code: replace `public/foo.mp4` → YouTube ID	Intially public → ref CDNs; `grep -r "public/et(singlewebm)src/"` must return 0	
5	Run build + e2e on Vercel preview; verify Lighthouse	Build & deploy green; perf budget passed	Revert commit
6	`git rm -r public/videos public/photos; git commit -m "Work iffted checks;"`	`du -sh .` shows new size	Tagize and reset
7	`git filter-repo --path public/ --invert-paths` (or BFG)	to given history; `git count-objects -vH` shows pack size	Force-push previous SHA backup
8	Clamp size: verify new clone ≤ 10 MB	`git clone --depth=1` works; perf budget	Restore from relog backup tag

7.6 Effectiveness and efficiency safeguards

Effectiveness — measure the bytes saved, not just removed: hiring the facade block cements an additional ~30% drop in initial page weight because YouTube iframe loads are deferred until interaction.

Efficiency — upload off-peak; batch through YouTube Data API v3 with `--max-concurrency=3`; record mappings in a CSV that checks into the repo so release managers can audit replacements without a GitHub history crawl. Final target: working tree < 10 MB; .git pack < 5 MB after filter-repo rewrite.

8. Audit Findings on Commits (Observation only)

No monetary conclusions are drawn from the commit log. The screenshots supplied show that one contributor (username beginning talaganaRajesh) authored a long sequence of fixes, integrations (payment gateway, iCarry delivery, OTP login), and admin-panel flows starting 22/12/2025, while the owner account vyraherbals-ux authored product-page and checkout revisions between 02/2026 and 05/2026. The visible commit graph indicates the contributor carried the broad surface area (auth, orders, iCarry diagnostics) while the owner focused on UI surface areas (hero, checkout, meta-pixel).

8.1 Mapping of observed commit themes to repo areas

Commit theme (sample messages)	Repo area touched
"backend apis added"; "cart functionality done"; "wishlist feature implemented"	app/api/cart, app/api/wishlist, src/Components/Contexts/CartContext
"auth implementation done"; "otp based phone auth done"; "auth fix"	src/Components/AuthContext
"order system, partially done, needs payment gateway"; "payment gateway payment booking displays buzz"	src/Components/orders
"icarry done"; "icarry fix"; "icarry unassignedorder created"; "extend src/Utils/orders"	src/Utils/orders
"blogs"; "reviews done"; "wishlist feature implemented"; "category"	src/Components and admin blogs/products pages

9. Prioritized Fix List

Apply in this order. Each row carries an owner-side "done when" criterion that is measurable, so review can stop being subjective.

Priority	Task	Done when
P0	Remove jwt fallback in adminAuth.ts; require env var; rotate token	CI grep finds zero hits on the literal; dev starts up cleanly
P0	Delete hard-coded admin pair; switch login to bcrypt + Supabase Auth	New auth flow passes manual penetration test on staging
P0	Lock /api/cart/* and /api/wishlist/* to authenticated session	curl tests without auth return 401 across all mutating endpoints
P0	Fix /api/orders/info so guest orders need signed token	Manual test with random order_id returns 403
P0	Restrict /api/data table arg to allow-list; require admin role	POST with `table=users` returns 403 unless admin
P0	Re-enable TypeScript checks in CI	Build fails on first type error
P1	Replace Date.now() order IDs with UUIDv4	Idempotency test: 1000 parallel inserts produce 1000 unique ids

P1	Re-fetch authoritative prices on /api/coupon/validate	Test tampered price payload returns discount 0
P1	Add global rate limit middleware (5 req/min unauth, 60 req/min auth)	Synthetic load test confirms slice enforced
P1	Strip PII console.logs; gate debug output behind env flag	Production bundle does not contain order/cart texts in source ma
P2	Refactor Checkout.tsx & Singleproducts.tsx	LOC drop $\geq 30\%$; per-component LOC ≤ 400
P2	Move admin metrics to RPC with date range + pagination	Dashboard query $\leq 250\text{ms}$ at 30k orders
P2	Dedup UI libs; lock engines.field in package.json	Bundle analyzer reports $\leq 350\text{kb}$ first load JS
P3	Add Sentry, CSP header, Permissions-Policy, sitemap, OG image	Lighthouse perf ≥ 90 , a11y ≥ 95

10. Appendix — Provenance

Numbers in this report trace to:

- Archive byte count: file metadata received when vyra-herbals-main.zip was delivered (= 86,184,097 bytes / 82.19 MB).
- Line counts: counted on-disk via `wc -l` during the extraction step (Checkout.tsx 1188, Singleproducts.tsx 1057, Header.tsx 886).
- API route count: file enumeration under `app/api/**` returned 63 handlers.
- Commit authors / themes: taken from supplied commit-history screenshots; no values were invented.

Open data gaps (treat as "not yet verified"):

- Live `du -sh public/*` was not re-run this turn because the original sandbox extraction had been cleared. The bucket table in §1.2 is a structural estimate; re-run on a fresh clone to firm up.
- A working `.git` history pack was not bundled with the archive, so pre-offload size & post-offload size from `git filter-repo` cannot be quoted yet; the sequence in §7.5 gives the reproducible recipe.